

DSA WITH JAVA

Concise Revision Notes

Complete coding journey — from basics to placement-level mastery. All 355 problems indexed by topic, with concept revision bullets and difficulty / pattern tags.

- 31 syllabus topics, in order
- 355 problems with difficulty + pattern tags
- Quick-revision concept bullets per topic
- Advanced DSA — Trees, Graphs, DP, Trie

Topic 00

Table of Contents

- 01 Basics of Programming
- 02 Control Flow Statements
- 03 Loops & Switch Statement
- 04 Methods and Arguments
- 05 Array Data Structures (Level 1)
- 06 Searching Algorithm
- 07 Sorting Algorithm (Basic)
- 08 Matrix (2D Arrays)
- 09 Time and Space Complexity
- 10 OOPS Introduction
- 11 String
- 12 Collection Framework
- 13 Memory Areas & Recursion - I
- 14 Mathematics for Programming
- 15 Arrays & ArrayList Problems (Level 2)
- 16 Sorting Algorithms (Advanced)
- 17 Binary Search
- 18 Hashing (Level 2)
- 19 Linked List
- 20 Stack & Queue
- 21 Number System (Bit Manipulation)
- 22 Greedy Approach & Problems
- 23 Recursion & Backtracking - Advanced
- 24 Binary Tree
- 25 Binary Search Tree (BST)
- 26 Balanced Trees (AVL, Red-Black)
- 27 Binary Heap
- 28 Sliding Window Technique
- 29 Graph

30 Dynamic Programming (DP)

31 Trie

Topic 01

Basics of Programming

Variables & Data Types

- Primitives
 - byte, short, int, long (long suffix l or L)
 - float, double (float suffix f or F; double's d/D optional)
 - boolean — true / false · char — UNICODE (0 – 65,535)
 - Every int literal is int by default · every float literal is double
- Non-Primitive — String, Arrays, Objects
 - String name = "Shery";
 - Stored as references on the stack, actual data lives in the heap

Input / Output

- Scanner class (java.util.Scanner)
 - nextInt(), nextLong(), nextDouble(), nextLine(), next()
 - Issue with nextLine() after nextInt() — consume the leftover newline
- System.out
 - print(), println(), printf()

Operators

- Arithmetic + – * / % · int / int yields int
- Relational < > <= >= == !=
- Logical && || !
- Unary ++ — (only on variables, never constants)
- Special — / shrinks number, % extracts last digit(s)
- Precedence & Associativity matter for complex expressions

Expressions & Formulas

- String + anything → concatenation
- int + int → arithmetic addition
- Mixed types → type promotion to wider type

Math API (`java.lang.Math`)

- `abs()`, `floor()`, `ceil()`, `round()`, `sqrt()`, `cbrt()`
- `pow(a,b)`, `max(a,b)`, `min(a,b)`, `random()`

Topic 01

Basics — Problems

1. Sum of two integers

Take two integers a & b with some constant values. Print their sum on the console.

Example 1 :

Input: a = 5, b = 7

Output: 12

2. Print message with values and sum

Take two integers a & b with some constant values. Print their sum on the console with the appropriate message and values.

Example 1 :

Input: a = 5, b = 7

Output: Sum of 5 & 7 is 12

3. Accept two integers and print sum

Take two integers a & b from the user. Print their sum on the console.

Example 1 :

Input: a = 5, b = 7

Output: 12

4. Accept name & age — formatted output

Accept String name and int age from the user. Print in the given format.

Example 1 :

Input: name = "Shery", age = 12

Output: Hello Shery, you are 12 years old.

5. Increment & Decrement practice

Practise ++ and -- (pre / post) and observe the variable values after each step.

Example 1 :

Input: a = 5; a++; ++a;

Output: Final a = 7

6. Swap two numbers

Accept two numbers and print them after swapping their values. Do it once with a third variable and once without.

Example 1 :

Input: a = 5.0, b = 3.0

Output: a = 3.0, b = 5.0

7. Rectangle — Area & Perimeter

Accept length and breadth of a rectangle. Print area and perimeter.

Example 1 :

Input: `l = 5, b = 3`

Output: `Area = 15, Perimeter = 16`

8. Compound Interest

Accept the principal, rate of interest and the time in years. Compute and print compound interest.

Example 1 :

Input: `P = 1000, R = 5, T = 3`

Output: `$157.63`

Constraints:

- Use `Math.pow()`

9. Area of Triangle — Heron's Formula

Accept the three sides `s1`, `s2`, `s3` of a triangle. Compute area using Heron's formula.

Example 1 :

Input: `s1 = 5, s2 = 7, s3 = 9`

Output: `Area: 17.412 sq units`

Topic 02

Control Flow Statements

if / if-else / else-if ladder

- if works only on boolean values
- else cannot exist without if; else has no condition of its own
- if-else-if = chained conditions evaluated top-to-bottom
- Nested if — if inside another if (any depth)

Ternary Operator

- `<condition> ? value_if_true : value_if_false`
- Compact replacement for simple if-else assignment

Type Casting / Type Conversion

- Implicit / Widening — smaller → larger (automatic)
- Explicit / Narrowing — larger → smaller; syntax (type) value
- Type Promotion — mixed expression promotes to widest operand type

Topic 02

Control Flow — Problems

10. Greatest between two numbers

Accept two numbers and determine which one is greater.

Example 1 :

Input: a = 9, b = 12

Output: 12

Example 2 :

Input: a = 100, b = 67

Output: 100

11. Gender-based greeting

Accept gender from the user. Print "Hello Sir" for male and "Hello Madam" for female.

Example 1 :

Input: gender = M

Output: Hello Sir

Example 2 :

Input: gender = F

Output: Hello Madam

12. Wrong gender input + validation

Extend the previous problem — for any input other than M / F, print "Invalid input".

Example 1 :

Input: gender = X

Output: Invalid input

13. Even or Odd

Accept an integer and determine whether it is even or odd.

Example 1 :

Input: a = 9

Output: 9 is an odd number

Example 2 :

Input: a = 24

Output: 24 is an even number

14. Voter eligibility (+ years left)

Accept name and age. A valid voter must be 18+. If eligible — "Hello [Name], you are a valid voter." Otherwise — "Hello [Name], you will be eligible to vote in (18–age) years."

Example 1 :

Input: name = Shery, age = 19

Output: Hello Shery, you are a voter.

Example 2 :

Input: name = Simon, age = 13

Output: Hello Simon, you will be eligible to vote in 5 years.

15. Day name from number (1–7)

Accept a day number 1–7 and print the corresponding day name. For out-of-range input print "WRONG INPUT".

Example 1 :

Input: day = 1

Output: Monday

16. Greatest among three numbers

Accept three numbers and print the greatest.

Example 1 :

Input: a = 4, b = 9, c = 6

Output: 9

17. Leap year check

A year is a leap year if it is divisible by 4, except for century years not divisible by 400.

Example 1 :

Input: year = 2024

Output: 2024 is a leap year

Example 2 :

Input: year = 1800

Output: 1800 is not a leap year

18. Shop discount

Apply slab discount: 0–5000 = 0%, 5001–7000 = 5%, 7001–9000 = 10%, >9000 = 20%.

Example 1 :

Input: amount = 4000

Output: 4000

Example 2 :

Input: amount = 6000

Output: 5700

19. Electricity bill

First 100 units @ 4.2 ; next 100 @ 6 ; next 200 @ 8 ; above 400 @ 13.

Example 1 :

Input: units = 150

Output: Total bill amount: 720

Example 2 :

Input: units = 700

Output: Total bill amount: 6520

Topic 03

Loops & Switch Statement

Need of loops

- Repeat a block of code until a condition holds
- Entry-controlled — for, while · Exit-controlled — do-while

for(init; condition; update)

- Body executes while condition is true
- Infinite forms — for(;;) · for(;;); · condition never returns false

while / do-while

- while — checks condition first (may execute zero times)
- do-while — executes body at least once, then checks condition
- Use when iteration count is unknown

Nested loops & Pattern Theory

- Loop inside loop — outer controls rows, inner controls columns
- How to identify nested loops — work out rows × cols mentally
- print() — same line · println() — newline at end

break & continue

- break — exits the nearest enclosing loop
- continue — skips to the next iteration

switch statement

- Allowed types — byte, short, char, int, String (not float / double — ambiguity)
- Cases must be unique · break required else fall-through
- Multiple cases can be combined with commas
- Java 14+ arrow form
 - case X -> ... ; no break required
 - Switch expression with yield keyword

Topic 03

Loops — Problems

20. Print "Hello World" n times

Accept an integer n and print hello world n times.

Example 1 :

Input: $n = 5$

Output: hello world (printed 5 times)

21. Natural numbers up to n

Print all natural numbers from 1 to n .

Example 1 :

Input: $n = 5$

Output: 1 2 3 4 5

Example 2 :

Input: $n = 10$

Output: 1 2 3 4 5 6 7 8 9 10

22. Reverse — n to 1

Print numbers from n down to 1.

Example 1 :

Input: $n = 5$

Output: 5 4 3 2 1

23. Multiplication table

Accept a number n and print its multiplication table up to $\times 10$.

Example 1 :

Input: $n = 7$

Output: 7 14 21 28 35 42 49 56 63 70

24. Sum up to n terms

Compute the sum of the first n natural numbers.

Example 1 :

Input: $n = 5$

Output: Sum = 15

Example 2 :

Input: $n = 10$

Output: Sum = 55

25. Sum of even & odd separately

Compute the sum of even numbers and the sum of odd numbers up to n .

Example 1 :

Input: $n = 10$

Output: Even = 30, Odd = 25

26. Print all factors of a number

Print all divisors of n.

Example 1 :

Input: n = 6

Output: 1 2 3 6

Example 2 :

Input: n = 18

Output: 1 2 3 6 9 18

27. Sum of all factors

Print the sum of all divisors of n.

Example 1 :

Input: n = 6

Output: 12

28. Prime number check

A prime is a positive integer > 1 with only two divisors — 1 and itself.

Example 1 :

Input: n = 17

Output: 17 is a prime number

Example 2 :

Input: n = 18

Output: 18 is not a prime number

29. Power of a number (a^b)

Compute a raised to the power b by repeated multiplication.

Example 1 :

Input: a = 2, b = 10

Output: 1024

30. Print each digit of a number

Print every digit of n on a new line (from left to right).

Example 1 :

Input: n = 1234

Output: 1\n2\n3\n4

31. Sum of digits

Compute the sum of the digits of n.

Example 1 :

Input: n = 1234

Output: 10

Example 2 :

Input: n = 897

Output: 24

32. Reverse a number

Print the digit-reversed version of n.

Example 1 :

Input: n = 12345

Output: 54321

33. Palindrome number check

n is palindrome if its reverse equals itself.

Example 1 :

Input: n = 121

Output: Palindrome

Example 2 :

Input: n = 12345

Output: Not palindrome

34. Strong number check

Strong number — sum of factorials of digits equals the number itself.

Example 1 :

Input: n = 145

Output: 145 is a strong number ($1! + 4! + 5! = 145$)

35. Automorphic number

n is automorphic if its square ends with n itself.

Example 1 :

Input: n = 25

Output: 25 is automorphic ($25^2 = 625$)

36. Repeat Hello (do-while)

Keep printing "Hello" until the user enters something other than 1.

Example 1 :

Input: input: 1, 1, 0

Output: Hello, Hello, exits

37. Menu-driven calculator (do-while)

Show add / sub / mul / div / exit menu in a loop until the user chooses exit.

Example 1 :

Input: choice = 1, a = 10, b = 5

Output: Sum = 15

38. Weekday using switch

Accept 1–7 and print the corresponding day name using a switch statement.

Example 1 :

Input: d = 3

Output: Wednesday

39. Vowel or consonant using switch

Use switch fall-through for vowels; otherwise consonant.

Example 1 :

Input: c = 'a'

Output: Vowel

Example 2 :

Input: c = 'k'

Output: Consonant

40. Project — Guess the Number

Generate a random number 1–100. Read user guesses in a loop and respond with "Too high", "Too low", or "Congrats".

Example 1 :

Input: target = 40, guesses = 23, 67, 40

Output: Too low → Too high → Congrats

Topic 03

Pattern Programming

41. Right Triangle — Star

Print a right-angled triangle of stars of side n.

Example 1 :

Input: n = 5

Output: *
* *
* * *
* * * *
* * * * *

42. Right Triangle — Number

Print a right-angled triangle of numbers — row i has 1..i.

Example 1 :

Input: n = 5

Output: 1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

43. Right Triangle — Alphabet

Print a right-angled triangle of alphabets — row i has A..(letter at position i).

Example 1 :

Input: n = 5

Output: A
A B
A B C
A B C D
A B C D E

44. Inverted Right Triangle

Print an inverted right-angled triangle of stars.

Example 1 :

Input: n = 5

Output: * * * * *\n* * * *\n* * *\n* *\n*

45. Mirrored Right Triangle

Right-aligned triangle of stars.

Example 1 :

Input: n = 5

Output: *\n * *\n * * *\n * * * *\n * * * * *

46. Triangle (Pyramid)

Centered triangle of stars.

Example 1 :

Input: n = 5

Output: *\n * *\n * * *\n * * * *\n * * * * *

47. V Pattern

Two diagonals of stars converging at the bottom centre.

Example 1 :

Input: n = 5

Output: * *\n * *\n * *\n * *\n *

48. X Pattern

Two crossing diagonals forming the letter X. n must be odd, $n \geq 3$.

Example 1 :

Input: n = 5

Output: * *\n * *\n * *\n * *\n *

Constraints:

- n $\geq$ 3
- n is an odd integer

Topic 04

Methods and Arguments

Method Signature / Prototype

- `returnType methodName(parameter list) { body }`
- Signature = method name + parameter list (used to resolve overloading)

Static vs Non-Static Methods

- static
 - Invoked using the class name
 - Cannot access instance fields / instance methods directly
- instance / non-static — Factory Method
 - Invoked using an object reference
 - Has access to both static and instance state

Argument Passing

- Formal vs Actual
 - Formal — parameters in the definition
 - Actual — values supplied at the call site
- Pass-by-value (Java)
 - Primitives pass a copy of the value
 - Objects pass a copy of the reference — internal mutations are visible

Varargs — `Type... name`

- Exactly three dots — variable length arguments
- Only one varargs per method
- If other params exist, varargs must be the last parameter

Method Invocation

- static — `ClassName.method(args)`
- instance — `obj.method(args)`

Topic 05

Array Data Structures (Level 1)

Why Arrays?

- Fixed-size, contiguous storage of same-type elements
- $O(1)$ indexed access — base address + index $\times$ element size
- Stack holds the reference · Heap holds the array object

Initialization

- `int[] arr = new int[size];`
- `int[] arr = new int[]{1, 2, 3};`
- `int[] arr = {1, 2, 3};`

Default Values

- `int / long` $\rightarrow$ 0
- `float / double` $\rightarrow$ 0.0
- `char` $\rightarrow$ `'\u0000'`
- `boolean` $\rightarrow$ `false`
- Non-primitive $\rightarrow$ `null`

Common Operations

- Traversal — `for` / `for-each`
- Reversal — two-pointer swap, $O(N)$ time, $O(1)$ space
- Rotation — left / right by 1 or K

Exceptions

- `ArrayIndexOutOfBoundsException`, `NullPointerException`

Topic 05

Arrays — Problems

49. Sum & average of array elements

Accept array nums of size n. Print sum and mean of all elements.

Example 1 :

Input: n = 5, nums = [10, 15, 20, 25, 30]

Output: Sum = 100, Mean = 20.0

50. Greatest element & its index

Find the greatest element in nums and print its index.

Example 1 :

Input: nums = [10, 5, 2, 37, 8]

Output: Greatest = 37 at index 3

51. Second greatest element

Find the second greatest element.

Example 1 :

Input: nums = [10, 5, 2, 37, 8]

Output: Second greatest = 10

52. Is array sorted?

Check if nums is sorted in non-decreasing order.

Example 1 :

Input: nums = [1, 2, 3, 4, 5]

Output: true

Example 2 :

Input: nums = [5, 2, 8, 4]

Output: false

53. Reverse copy of array

Create a duplicate array with elements reversed.

Example 1 :

Input: `nums = [1, 2, 3, 4, 5]`

Output: `[5, 4, 3, 2, 1]`

54. Left rotate by 1

Perform a single-step left rotation.

Example 1 :

Input: `nums = [1, 2, 3, 4, 5]`

Output: `[2, 3, 4, 5, 1]`

55. Left rotate by K

Left rotate `nums` by `K` positions.

Example 1 :

Input: `nums = [1, 2, 3, 4, 5], K = 2`

Output: `[3, 4, 5, 1, 2]`

56. Reverse array in-place

Reverse the array without using extra space (two-pointer).

Example 1 :

Input: `nums = [1, 2, 3, 4, 5]`

Output: `[5, 4, 3, 2, 1]`

57. Move negatives left, positives right

Rearrange nums so all negatives sit to the left and positives to the right (any order).

Example 1 :

Input: nums = [-1, 2, -3, -4, 5]

Output: [-1, -3, -4, 2, 5]

58. Count subarrays with given sum

Count contiguous subarrays whose elements sum to target.

Example 1 :

Input: nums = [1,2,3,7,5], target = 12

Output: 2 → [2,3,7] and [7,5]

Topic 06

Searching Algorithm

Linear Search

- Walk the array — return index when $\text{arr}[i] == \text{key}$, else -1
- Time $O(N)$ · Space $O(1)$ · works on unsorted data

Binary Search

- Requires sorted data
- Maintain lo, hi · $\text{mid} = \text{lo} + (\text{hi} - \text{lo}) / 2$
- If $\text{arr}[\text{mid}] == \text{key} \rightarrow \text{return}$; if $< \text{key} \rightarrow \text{lo} = \text{mid} + 1$; else $\text{hi} = \text{mid} - 1$
- Time $O(\log N)$ · Space $O(1)$ iterative
- Avoid $(\text{lo} + \text{hi})/2$ — risk of integer overflow

Practice problems for Binary Search live in Topic 17 (Binary Search).

Topic 07

Sorting Algorithm (Basic)

Stable vs Unstable

- Stable — preserves order of equal keys (Bubble, Insertion, Merge)
- Unstable — may reorder equal keys (Selection, Quick, Heap)

Comparison-based sorting

- Decisions taken via comparisons ($<$ / $>$)
- Lower bound — $\Omega(N \log N)$ for comparison sorts

In-place sorting

- Sort within input array using $O(1)$ extra space
- Bubble, Selection, Insertion are all in-place

Quick recap

- Bubble — $O(N^2)$ worst · $O(N)$ best · stable · in-place
- Insertion — $O(N^2)$ worst · $O(N)$ best · stable · in-place
- Selection — $O(N^2)$ all cases · unstable · in-place

Topic 07

Basic Sorting — Problems

59. Bubble Sort

Sort nums in ascending order using bubble sort. Repeatedly swap adjacent out-of-order pairs.

Example 1 :

Input: nums = [4, 3, 6, 7, 11, 9]

Output: [3, 4, 6, 7, 9, 11]

60. Insertion Sort

Sort nums by repeatedly inserting each element into the sorted prefix.

Example 1 :

Input: nums = [4, 3, 6, 7, 11, 9]

Output: [3, 4, 6, 7, 9, 11]

61. Selection Sort

Sort nums by repeatedly selecting the minimum from the unsorted suffix and swapping it to the front.

Example 1 :

Input: nums = [4, 3, 6, 7, 11, 9]

Output: [3, 4, 6, 7, 9, 11]

Topic 08

Matrix (2D Arrays)

2D arrays — array of arrays

- `int[][] m = new int[rows][cols];`
- `m.length` → rows · `m[i].length` → columns of row `i`

Jagged arrays

- Rows can have different lengths
- Must supply the first-dimension size at creation

Traversal patterns

- Row-major — for `i`, for `j`
- Column-major, Diagonal, Spiral

Common operations

- Sum of diagonals, transpose, rotation, flipping
- Search in a 2D matrix — binary search on flattened view

Exceptions

- `ArrayIndexOutOfBoundsException`, `NullPointerException`

Topic 08

Matrix — Problems

62. Matrix Diagonal Sum

Compute the sum of both diagonals of a square matrix (don't double-count the centre).

Example 1 :

Input: `[[1, 2, 3], [4, 5, 6], [7, 8, 9]]`

Output: `25`

63. Transpose Matrix

Return the transpose of an $m \times n$ matrix.

Example 1 :

Input: `[[1, 2, 3], [4, 5, 6], [7, 8, 9]]`

Output: `[[1, 4, 7], [2, 5, 8], [3, 6, 9]]`

64. Flipping an Image

Flip each row horizontally then invert ($0 \leftrightarrow 1$).

Example 1 :

Input: `[[1, 1, 0], [1, 0, 1], [0, 0, 0]]`

Output: `[[1, 0, 0], [0, 1, 0], [1, 1, 1]]`

65. Rotate Image

Rotate an $n \times n$ matrix 90° clockwise in-place (transpose + reverse each row).

Example 1 :

Input: `[[1, 2, 3], [4, 5, 6], [7, 8, 9]]`

Output: `[[7, 4, 1], [8, 5, 2], [9, 6, 3]]`

66. Search a 2D Matrix

Matrix is sorted row-wise and the first integer of each row > last of previous row. Binary search.

Example 1 :

Input: `m = [[1,3,5],[7,9,11]], target = 9`

Output: `true`

67. Set Matrix Zeroes

If any element is 0, set its entire row and column to 0 — in-place.

Example 1 :

Input: `[[1,1,1],[1,0,1],[1,1,1]]`

Output: `[[1,0,1],[0,0,0],[1,0,1]]`

Topic 09

Time and Space Complexity

Time Complexity

- Not wall-clock time — count of steps as a function of N
- Best < Average < Worst (usually report Worst)
- Ignore lower-order terms and constants for large N

Big-O Notations

- Big-O $O(N)$ — Upper bound (Worst case)
- Theta $\theta(N)$ — Tight bound (Average)
- Omega $\Omega(N)$ — Lower bound (Best case)

Growth order (slowest → fastest)

- $O(1) < O(\log N) < O(N) < O(N \log N) < O(N^2) < O(2^N) < O(N!)$
- constant < log < linear < quadratic < exponential < factorial

TLE & Constraints

- Typical machine does $\sim 10^8$ simple operations / second
- Match algorithm complexity to input-size constraint

Space Complexity

- Memory used by inputs + auxiliary space (extra arrays, recursion stack)
- Local primitive variables don't count as extra space

Topic 10

OOPS Introduction

Class & Object

- Class — blueprint · Object — instance in heap memory
- Class has attributes (fields) + behaviour (methods)
- Each new object has its own copy of instance fields

Constructor

- Runs once after object creation
- Default (no-arg) and parameterised constructors
- Used to initialise instance fields

this keyword

- Refers to the current calling object
- Disambiguates field vs parameter name

Encapsulation (basics)

- Make data members private; expose via getters / setters
- Validates state changes and hides implementation

Topic 10

OOPS — Problems

68. Design a Private Cab Service

Model a cab booking with fields like driverName, carNumber, fare. Add methods to compute trip cost.

Example 1 :

Input: Trip(distance = 12, ratePerKm = 14)

Output: Fare = 168

69. Design a Railway Ticket Class

Model a ticket with pnr, passengerName, trainNo, source, destination. Add a print() that shows the ticket.

Example 1 :

Input: Ticket(name = Shery, train = 12345)

Output: Ticket printed

Topic 11

String

Literals & Immutability

- String name = "Adarsh" — stored in the String Constant Pool (heap)
- Strings are immutable — every modification creates a new object

Constructors

- String() , String(String s) , String(char[] chars)

Important Methods

- length(), charAt(i), substring(i, j)
- indexOf(), lastIndexOf(), contains(), startsWith(), endsWith()
- toUpperCase(), toLowerCase(), trim(), strip(), replace()
- split(regex), toCharArray(), String.valueOf(), String.join()

Comparing Strings

- Avoid '==' (compares references) — use equals() / equalsIgnoreCase()
- compareTo() — lexicographic order (returns int)

Character operations

- Character.isLetter / isDigit / isWhitespace
- Character.toUpperCase / toLowerCase
- char arithmetic — 'a' - 'a' = 0 (use as index for 26-size arrays)

Topic 11

String — Problems

70. Print each character of string

Print each character of str on a new line.

Example 1 :

Input: str = "hello"
Output: h\n e\n l\n l\n o\n

71. Reverse a string

Print str in reverse order.

Example 1 :

Input: str = "hello"
Output: "olleh"

72. Palindrome — two pointer

A palindrome reads the same forwards and backwards. Use a two-pointer check.

Example 1 :

Input: str = "madam"
Output: true

Example 2 :

Input: str = "shery"
Output: false

73. Count vowels & consonants

Count vowels, consonants and spaces in str.

Example 1 :

Input: str = "hello world"
Output: Vowels: 3, Consonants: 7, Spaces: 1

74. Toggle characters

Convert each lowercase → uppercase and vice versa.

Example 1 :

Input: `str = "AcgDfD"`

Output: `"aCGdFd"`

75. Count words with given prefix

Count how many strings in words start with the prefix pref.

Example 1 :

Input: `words = [pay, attire, practice, attend], pref = at`

Output: `2`

76. Capitalize first & last char of each word

For every word in the sentence, capitalise the first and last alphabet.

Example 1 :

Input: `"hello world"`

Output: `"Hello World"`

77. Maximum number of words in a sentence

Given an array of sentences, return the maximum word count among them.

Example 1 :

Input: `["hello world", "a b c d"]`

Output: `4`

78. Frequency of each character

Print the count of every character in str.

Example 1 :

Input: str = "hello"

Output: h: 1, e: 1, l: 2, o: 1

79. Check anagram

Two strings are anagrams if one is a rearrangement of the other.

Example 1 :

Input: s = anagram, t = nagaram

Output: true

Example 2 :

Input: s = naruto, t = tanjiro

Output: false

80. Sort the words of a sentence

Sort words of the sentence in lexicographic order and print the new sentence.

Example 1 :

Input: "ek machli paani me gayi chapaak"

Output: "chapaak ek gayi machli me paani"

81. Rotate string

Check whether s2 is a rotation of s1.

Example 1 :

Input: s1 = abcde, s2 = cdeab

Output: true

Topic 12

Collection Framework

Collection vs Collections

- Collection — root interface (List, Set, Queue extend it)
- Collections — utility class with static helpers (sort, reverse, ...)

List family

- ArrayList — dynamic array, $O(1)$ random access
- LinkedList — doubly linked list, $O(1)$ ends

Set family

- HashSet — unique elements · $O(1)$ average
- LinkedHashSet — preserves insertion order
- TreeSet — sorted · $O(\log N)$ operations

Map family

- HashMap<K,V> — key-value pairs, $O(1)$ average
- LinkedHashMap — preserves insertion order
- TreeMap — sorted keys · $O(\log N)$ operations

Hashing basics

- hashCode() decides bucket · equals() resolves collisions
- Override both together for custom keys

Topic 12

Collections — Problems

82. Print unique elements in array

Print all elements that occur exactly once (use HashSet / HashMap).

Example 1 :

Input: nums = [1, 2, 2, 3, 4, 4]

Output: [1, 3]

83. Frequency of elements in array

Print each element with its frequency.

Example 1 :

Input: nums = [1, 2, 2, 3, 4, 4]

Output: 1: 1, 2: 2, 3: 1, 4: 2

84. Jewels and Stones

Count characters of S that appear in J.

Example 1 :

Input: J = "aA", S = "aAAbbbb"

Output: 3

85. Check if sentence is pangram

Pangram contains every letter a–z at least once.

Example 1 :

Input: "The quick brown fox jumps over the lazy dog"

Output: true

86. First letter to appear twice

Return the first letter that appears twice while scanning left-to-right.

Example 1 :

Input: `s = "abccbaacz"`

Output: `'c'`

87. Sum of unique elements

Return the sum of elements that occur exactly once.

Example 1 :

Input: `[1, 2, 3, 2]`

Output: `4`

88. Sort the People

Given `names[]` and `heights[]`, return names sorted by height descending.

Example 1 :

Input: `names = [Mary, John, Emma], heights = [180, 165, 170]`

Output: `[Mary, Emma, John]`

89. Most Frequent Even Element

Return the most frequent even element; tie-breaker — smallest value.

Example 1 :

Input: `nums = [0, 1, 2, 2, 4, 4, 1]`

Output: `2`

Topic 13

Memory Areas & Recursion - I

JVM memory layout

- Stack — method frames, locals, parameters
- Heap — all objects and class instances
- Method area — class metadata, static fields, bytecode

Stack memory & stack frames

- Each method call pushes a new frame
- Frame stores locals + return address
- Frame is popped on return — locals are lost
- Excessive recursion → StackOverflowError

Recursion basics

- Function that calls itself on a smaller / simpler input
- Recursive solution = base case + recursive case
- Recurrence relation describes time complexity

Base case

- Smallest input the function can answer directly
- Without it → infinite recursion → stack overflow

Recursive call mechanics

- Trust the recursion — assume the sub-call returns the right answer
- Build the answer for current input from the sub-call's result

Topic 13

Recursion I — Problems

90. Print name N times

Print the same name n times using recursion.

Example 1 :

Input: name = Shery, n = 3

Output: Shery Shery Shery

91. Print 1 to N

Print numbers 1 .. n in order using recursion.

Example 1 :

Input: n = 5

Output: 1 2 3 4 5

92. Print N to 1

Print numbers n .. 1 in order using recursion.

Example 1 :

Input: n = 5

Output: 5 4 3 2 1

93. Sum of first N numbers

Compute $1 + 2 + \dots + n$ using recursion.

Example 1 :

Input: n = 5

Output: 15

94. Factorial of a number

Compute $n!$ recursively.

Example 1 :

Input: $n = 5$

Output: 120

95. Reverse an array (recursion + two pointer)

Reverse nums in-place using recursion.

Example 1 :

Input: `nums = [1, 2, 3, 4, 5]`

Output: `[5, 4, 3, 2, 1]`

96. Check palindrome string (recursive)

Use two-pointer recursion to verify palindrome.

Example 1 :

Input: `str = madam`

Output: `true`

97. Fibonacci number (tree recursion)

$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$; $\text{fib}(0)=0$, $\text{fib}(1)=1$.

Example 1 :

Input: $n = 8$

Output: 21

Topic 14

Mathematics for Programming

GCD / HCF — Euclidean Algorithm

- $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$ · $\text{gcd}(a, 0) = a$
- Time $O(\log \min(a, b))$

LCM via GCD

- $\text{lcm}(a, b) = (a \times b) / \text{gcd}(a, b)$
- Compute gcd first to avoid overflow

Prime & Factors

- Trial division up to $\sqrt{n}$ — $O(\sqrt{n})$
- All primes > 3 are of form $6k \pm 1$

Sieve of Eratosthenes

- Mark multiples of each prime starting at p^2 up to n
- Time $O(n \log \log n)$ · Space $O(n)$

Fast Exponentiation

- $\text{pow}(a, b) = \text{pow}(a, b/2)^2$ if b even
- $\text{pow}(a, b) = \text{pow}(a, b/2)^2 \cdot a$ if b odd
- Time $O(\log b)$

Logarithmic operations

- Number of digits = $\text{floor}(\log_{10}(n)) + 1$

Topic 14

Mathematics — Problems

98. GCD / HCF of two numbers

Compute $\text{gcd}(a, b)$ using Euclidean algorithm.

Example 1 :

Input: $a = 36, b = 24$

Output: 12

99. GCD of array

Reduce the array using gcd pairwise.

Example 1 :

Input: [12, 18, 24]

Output: 6

100. LCM of two numbers

$\text{lcm} = a \cdot b / \text{gcd}(a, b)$.

Example 1 :

Input: $a = 4, b = 6$

Output: 12

101. Count digits of a number

Use $\log_{10}$ trick.

Example 1 :

Input: $n = 12345$

Output: 5

102. Median of data

Sort and pick middle (average of two middles if even).

Example 1 :

Input: [1, 3, 5, 2, 4]

Output: 3

103. Palindromic number

Number is palindrome if reverse == original.

Example 1 :

Input: n = 121

Output: true

104. Factors of a number ($\sqrt{n}$)

List divisors using i and n/i up to $\sqrt{n}$.

Example 1 :

Input: n = 36

Output: 1 2 3 4 6 9 12 18 36

105. Prime number check ($\sqrt{n}$)

Trial division up to $\sqrt{n}$.

Example 1 :

Input: n = 17

Output: Prime

106. Count primes — Sieve of Eratosthenes

Count primes $< n$ using sieve.

Example 1 :

Input: $n = 10$

Output: $4 \rightarrow 2, 3, 5, 7$

107. Kth factor of n

Print the K-th smallest divisor of n.

Example 1 :

Input: $n = 12, K = 3$

Output: 3

108. Integer square root

Binary search for the largest i with $i \cdot i \leq n$.

Example 1 :

Input: $n = 50$

Output: 7

109. Power (x^n) — Fast Exponentiation

Compute x^n in $O(\log n)$.

Example 1 :

Input: $x = 2, n = 10$

Output: 1024

110. Power of Two

n is power of 2 iff $n > 0$ and $(n \& (n - 1)) == 0$.

Example 1 :

Input: $n = 16$

Output: true

Example 2 :

Input: $n = 18$

Output: false

Topic 15

Arrays & ArrayList Problems (Level 2)

ArrayList basics

- Dynamic array — grows automatically (typical 1.5× growth)
- `add(e)`, `add(idx, e)`, `get(idx)`, `set(idx, e)`, `remove(idx)`, `size()`

Two Pointer

- Two indices moving toward each other or in the same direction
- Turns nested loops into $O(N)$ on sorted arrays

Prefix Sum

- $\text{prefix}[i] = a[0] + \dots + a[i]$
- Range sum $[l..r] = \text{prefix}[r] - \text{prefix}[l-1]$ in $O(1)$

Sliding Window — intro

- Window $[l..r]$ — expand r , contract l on violation
- Fixed and variable size flavours (see Topic 28)

Kadane's Algorithm

- $\text{currMax} = \max(a[i], \text{currMax} + a[i])$
- Track `globalMax` over all i — $O(N)$, $O(1)$

Topic 15

Arrays Level 2 — Problems

- 111. Max Consecutive Ones [Easy] — Sliding Window
- 112. Merge Sorted Array [Easy] — Two Pointer
- 113. Remove Duplicates from Sorted Array [Easy] — Two Pointer
- 114. Duplicate Zeros [Easy] — Simulation
- 115. Valid Mountain Array [Easy] — Traversal
- 116. Replace Elements with Greatest on Right [Easy] — Reverse Traversal
- 117. Squares of a Sorted Array [Easy] — Two Pointer
- 118. Move Zeroes [Easy] — Two Pointer
- 119. Sort Array By Parity [Easy] — Partition / Two Pointer
- 120. Fibonacci Number [Easy] — DP / Iteration
- 121. Two Sum [Easy] — Hashing
- 122. Running Sum of 1D Array [Easy] — Prefix Sum
- 123. Find Pivot Index [Easy] — Prefix Sum
- 124. Product of Array Except Self [Medium] — Prefix + Suffix
- 125. Maximum Subarray (Kadane's) [Medium] — Kadane
- 126. Majority Element (Boyer-Moore) [Medium] — Voting Algorithm
- 127. 3Sum [Medium] — Sorting + Two Pointer
- 128. Best Time to Buy & Sell Stock [Easy] — Kadane / Greedy
- 129. Best Time to Buy & Sell Stock II [Medium] — Greedy
- 130. Trapping Rain Water [Hard] — Two Pointer
- 131. Container With Most Water [Medium] — Two Pointer

Topic 16

Sorting Algorithms (Advanced)

Merge Sort

- Divide & Conquer — split, sort each half, merge
- Time $O(N \log N)$ all cases · Space $O(N)$
- Stable · preferred for linked lists

Quick Sort

- Choose pivot → partition → recurse
- Best / Avg $O(N \log N)$ · Worst $O(N^2)$ (bad pivot)
- In-place · not stable

Cyclic Sort

- When array contains numbers from a known range ($1..N$ or $0..N-1$)
- Place each value at its correct index by swapping
- Time $O(N)$ · Space $O(1)$ — perfect for missing / duplicate problems

Topic 16

Advanced Sorting — Problems

- 132. Missing Number [Easy] — Cyclic Sort
- 133. Find All Numbers Disappeared in Array [Easy] — Cyclic Sort
- 134. Set Mismatch [Easy] — Cyclic Sort
- 135. Find All Duplicates in an Array [Medium] — Cyclic Sort
- 136. Find the Duplicate Number [Medium] — Cycle Detection
- 137. First Missing Positive [Hard] — Cyclic Sort

Topic 17

Binary Search

Iterative & Recursive forms

- Iterative — while ($lo \leq hi$) loop
- Recursive — function on the smaller half
- Time $O(\log N)$ · Space $O(1)$ iterative / $O(\log N)$ recursive

Search-space reduction

- Each comparison eliminates half of the remaining range
- Works on any monotonic predicate, not only sorted arrays

Binary Search on Answer

- Define $[lo, hi]$ as the range of possible answers
- Use feasibility check $f(mid)$ to shrink the range
- Classic for 'minimum X such that condition holds' problems

Common variants

- Lower / Upper bound (first / last occurrence)
- Ceil / Floor, Search in rotated / mountain array

Topic 17

Binary Search — Problems

- 138. Binary Search on descending array [Easy] — Binary Search
- 139. Order Agnostic Binary Search [Easy] — Binary Search
- 140. Guess Number Higher or Lower [Easy] — Binary Search
- 141. Search Insert Position [Easy] — Binary Search
- 142. Ceiling of a number [Easy] — Binary Search
- 143. Floor of a number [Easy] — Binary Search
- 144. Smallest letter greater than target [Easy] — Binary Search
- 145. Valid Perfect Square [Easy] — Binary Search
- 146. First Bad Version [Easy] — Binary Search
- 147. First & Last Position in Sorted Array [Medium] — Binary Search
- 148. Count of element in sorted array [Medium] — Binary Search
- 149. Peak in Mountain / Bitonic array [Medium] — Binary Search
- 150. Search in Mountain / Bitonic array [Medium] — Binary Search
- 151. Minimum in Rotated Sorted Array [Medium] — Binary Search
- 152. Search in Rotated Sorted Array [Medium] — Binary Search
- 153. Search in Infinite Sorted Array [Medium] — Binary Search
- 154. Book Allocation Problem [Hard] — BS on Answer
- 155. Capacity to Ship Packages [Hard] — BS on Answer
- 156. Minimum Days to Make Bouquets [Hard] — BS on Answer
- 157. Smallest Divisor Given Threshold [Hard] — BS on Answer
- 158. Koko Eating Bananas [Hard] — BS on Answer
- 159. Split Array Largest Sum [Hard] — BS on Answer
- 160. Aggressive Cows [Hard] — BS on Answer

Topic 18

Hashing (Level 2)

Hashing fundamentals

- Average $O(1)$ insert / search / delete
- Hash function maps key $\rightarrow$ bucket index
- Trade-off — extra space for speed

Collision handling

- Separate chaining — list / tree per bucket
- Open addressing — linear / quadratic probing, double hashing

HashSet vs HashMap usage

- Set — uniqueness, fast contains()
- Map — frequency counts, look-ups, prefix-sum + map for subarray sums

Common idioms

- Prefix sum + map $\rightarrow$ count subarrays with given sum
- Frequency map $\rightarrow$ anagrams, most-frequent elements
- Sliding window + map $\rightarrow$ variable-window problems

Topic 18

Hashing — Problems

- 161. Intersection of Two Arrays [Easy] — HashSet
- 162. Intersection of Two Arrays II [Easy] — HashMap
- 163. Subarray Sum Equals K (Positive) [Easy] — Sliding Window
- 164. Subarray Sum Equals K [Medium] — Prefix Sum + HashMap
- 165. Subarray with 0 Sum [Medium] — Prefix Sum + HashSet
- 166. Longest Subarray with Sum K [Medium] — Prefix Sum + HashMap
- 167. Contiguous Array [Medium] — Prefix Sum + HashMap
- 168. Longest Consecutive Sequence [Medium] — HashSet
- 169. Count distinct elements in every window [Medium] — Sliding Window + HashMap
- 170. Insert Delete GetRandom O(1) [Hard] — HashMap + Design

Topic 19

Linked List

Node structure

- Singly — { value, next }
- Doubly — { value, prev, next }
- Circular — last.next points to head

Why Linked List?

- Dynamic memory — grow / shrink without resizing
- Insertion / deletion at known node is $O(1)$
- No random access — traversal is $O(N)$

Core operations

- Traversal, size, search
- Insert / delete at first, last, given index
- Reverse — iterative (3-pointer) and recursive

Two-Pointer (Slow & Fast)

- Middle of list — fast moves $2\times$ while slow moves $1\times$
- Cycle detection — Floyd's tortoise & hare
- N-th node from end — keep a gap of N then advance together

Use cases

- Stack / Queue implementations, music player, image gallery, DMA

Topic 19

Linked List — Problems

- 171. Design Linked List [Easy] — Implementation
- 172. Middle of Linked List [Easy] — Two Pointer (Tortoise & Hare)
- 173. Merge Two Sorted Lists [Easy] — Two Pointer
- 174. Linked List Cycle [Easy] — Cycle Detection (Floyd's)
- 175. Linked List Cycle II [Medium] — Cycle Detection
- 176. Reverse Linked List [Easy] — Iterative / Recursion
- 177. Convert Binary LL to Integer [Easy] — Traversal
- 178. Merge K Sorted Lists [Hard] — Heap / Divide & Conquer
- 179. Remove Duplicates from Sorted List [Easy] — Traversal
- 180. Remove Duplicates II [Medium] — Traversal
- 181. Intersection of Two Linked Lists [Easy] — Two Pointer
- 182. Remove Linked List Elements [Easy] — Traversal
- 183. Palindrome Linked List [Easy] — Two Pointer
- 184. Delete Node in Linked List [Easy] — Pointer Manipulation
- 185. Reverse Nodes in K Group [Hard] — List Manipulation
- 186. Swap Nodes in Pairs [Medium] — Pointer Manipulation
- 187. Remove Nth Node From End [Medium] — Two Pointer
- 188. Add Two Numbers [Medium] — Simulation
- 189. Copy List with Random Pointer [Hard] — HashMap / Cloning
- 190. Flattening a Linked List [Hard] — Recursion / Divide & Conquer

Topic 20

Stack & Queue

Stack — LIFO

- Operations — push, pop, peek ($O(1)$) · underflow / overflow on bounds
- Implementation — array, linked list, `java.util.Stack` / `Deque`
- Use cases — function-call stack, undo, balanced parentheses

Queue — FIFO

- Operations — offer / add, poll / remove, peek ($O(1)$)
- Implementation — `LinkedList`, `ArrayDeque` (preferred), circular array
- Use cases — FCFS scheduling, keyboard / printer buffers

Deque (Doubly-Ended Queue)

- Insert / remove at both ends — acts as stack and queue
- `offerFirst`, `offerLast`, `pollFirst`, `pollLast`, `peekFirst`, `peekLast`

Expression evaluation

- Infix $\rightarrow$ Postfix / Prefix with an operator stack and precedence rules
- Evaluate Postfix / Prefix using a single value stack

Monotonic Stack pattern

- Stack kept in increasing or decreasing order
- Backbone for Next Greater / Smaller / Stock Span / Histogram

Topic 20

Stack & Queue — Problems

- 191. Implement Queue using Stacks [Easy] — Simulation
- 192. Implement Stack using Queues [Easy] — Simulation
- 193. Infix to Postfix Conversion [Medium] — Stack (Expression)
- 194. Prefix to Infix Conversion [Medium] — Stack (Expression)
- 195. Prefix to Postfix Conversion [Medium] — Stack (Expression)
- 196. Postfix to Prefix Conversion [Medium] — Stack (Expression)
- 197. Postfix to Infix Conversion [Easy] — Stack (Expression)
- 198. Infix to Prefix Conversion [Medium] — Stack (Expression)
- 199. Next Larger Element [Easy] — Monotonic Stack
- 200. Smaller on Left [Easy] — Monotonic Stack
- 201. Next Greater Element I [Easy] — Monotonic Stack
- 202. Smallest number on left [Easy] — Monotonic Stack
- 203. Next Smaller Element [Easy] — Monotonic Stack
- 204. Backspace String Compare [Medium] — Stack / Two Pointer
- 205. Stock Span Problem [Medium] — Monotonic Stack
- 206. Online Stock Span [Medium] — Monotonic Stack
- 207. Min Stack [Medium] — Stack Design
- 208. Largest Rectangle in Histogram [Hard] — Monotonic Stack
- 209. Maximal Rectangle [Hard] — Stack + Matrix
- 210. Celebrity Problem [Medium] — Stack / Two Pointer
- 211. LRU Cache [Hard] — HashMap + DLL
- 212. LFU Cache [Hard] — HashMap + Design

Topic 21

Number System (Bit Manipulation)

Number Bases

- Binary (0b...), Octal (0...), Decimal, Hexadecimal (0x...)
- Decimal $\leftrightarrow$ Binary via repeated division / multiplication

Bitwise Operators

- AND $\&$ — 1 only when both bits are 1
- OR $|$ — 1 when at least one bit is 1
- XOR $\wedge$ — 1 when bits differ · $a \wedge a = 0$, $a \wedge 0 = a$
- NOT $\sim$ — inverts every bit
- Left shift $\ll$ — multiply by 2 · Right shift $\gg$ — divide by 2

Useful Tricks

- Check i-th bit: $(n \gg i) \& 1$
- Set i-th bit: $n | (1 \ll i)$
- Clear i-th bit: $n \& \sim(1 \ll i)$
- Toggle i-th bit: $n \wedge (1 \ll i)$
- Power of 2: $n > 0 \&\& (n \& (n - 1)) == 0$
- Swap without temp: $a \wedge= b; b \wedge= a; a \wedge= b;$

Topic 21

Bit Manipulation — Problems

- 213. Introduction to Bits and Tricks [Easy] — Bit Basics
- 214. Check if i-th bit is set or not [Easy] — Bit Masking
- 215. Check if number is odd or not [Easy] — Bit Check
- 216. Check if number is power of 2 [Easy] — Bit Trick
- 217. Count number of set bits [Easy] — Bit Counting
- 218. Set / Unset rightmost unset bit [Easy] — Bit Manipulation
- 219. Swap two numbers [Easy] — XOR Trick
- 220. Divide two numbers without * and / [Medium] — Bit Manipulation
- 221. Minimum bit flips to convert number [Medium] — XOR
- 222. Single Number - I [Medium] — XOR
- 223. Power Set (Bit Manipulation) [Medium] — Bit Masking
- 224. XOR of numbers in a range [Medium] — Pattern + XOR
- 226. Single Number - III [Medium] — XOR + Bit Masking

Topic 22

Greedy Approach & Problems

Greedy choice property

- Make the locally optimal choice at every step
- Hope (or prove) that local optima lead to the global optimum

Local vs Global optimum

- Greedy works only when local choices stay globally optimal
- Otherwise reach for DP / Backtracking

Common Strategies

- Sort input by some key (ratio, deadline, start, end...)
- Use a heap / priority queue when next choice depends on best-so-far
- Activity selection — earliest finishing time first
- Fractional Knapsack — sort by value / weight ratio
- Interval problems — sort by start / end then sweep

How to spot a greedy problem

- Optimisation problem (max / min / count)
- Choices can be ranked by a simple key
- No backtracking needed once a choice is made

Topic 22

Greedy — Problems

- 227. Assign Cookies [Easy] — Greedy + Sorting
- 228. Fractional Knapsack [Medium] — Greedy (Ratio Based)
- 229. Lemonade Change [Easy] — Greedy (Simulation)
- 230. Valid Parenthesis Checker [Hard] — Stack (Validation)
- 231. N Meetings in One Room [Medium] — Greedy (Activity Selection)
- 232. Jump Game I [Easy] — Greedy
- 233. Jump Game II [Medium] — Greedy
- 234. Minimum Platforms [Medium] — Greedy + Sorting
- 235. Job Sequencing Problem [Medium] — Greedy + Heap
- 236. Candy [Hard] — Greedy
- 237. Shortest Job First [Medium] — Greedy (Scheduling)
- 238. LRU Page Replacement [Medium] — Greedy / Simulation
- 239. Insert Interval [Medium] — Greedy + Intervals
- 240. Merge Intervals [Medium] — Greedy + Sorting

Topic 23

Recursion & Backtracking — Advanced

Tail vs Head Recursion

- Tail — recursive call is the last action; loop-convertible
- Head — work happens after the recursive call returns

Recursion tree

- Visualise calls as a tree; leaves are base cases
- Count of leaves / depth → time and space estimates

Backtracking template

- Choose → Explore → Un-choose (undo state on the way back)
- Prune impossible branches as early as possible

Classic Patterns

- Subsets / Power Set — include or exclude each element
- Permutations — fix-then-recurse with a visited / swap array
- Combinations — pick from index i to maintain order
- Grid / Maze — try 4 directions, mark visited, undo on backtrack
- Constraint satisfaction — Sudoku, N-Queens, Word Search

Topic 23

Backtracking — Problems

- 241. K-th Symbol in Grammar [Medium] — Recursion
- 242. Tower of Hanoi [Medium] — Recursion
- 243. Josephus Problem [Medium] — Recursion
- 244. Find Winner of Circular Game [Medium] — Simulation / Recursion
- 245. Sort List / Stack (Recursive) [Medium] — Recursion
- 246. Power Set [Medium] — Backtracking
- 247. Subsets [Medium] — Backtracking
- 248. Permutations [Medium] — Backtracking
- 249. Subsets II [Medium] — Backtracking
- 250. Letter Combinations of Phone Number [Medium] — Backtracking
- 251. Combination Sum [Medium] — Backtracking
- 252. Combination Sum III [Medium] — Backtracking
- 253. Rat in a Maze [Medium] — Backtracking
- 254. Sudoku Solver [Hard] — Backtracking
- 255. N-Queens [Hard] — Backtracking
- 256. Word Search [Medium] — Backtracking
- 257. Permutation Sequence [Hard] — Backtracking + Math

Topic 24

Binary Tree

Terminology

- Root, parent, child, sibling, leaf, ancestor, descendant
- Height, depth, level, diameter
- Full / Complete / Perfect / Balanced / Skewed trees

Node Structure

- `class Node { int val; Node left, right; }`
- Max 2 children per node

Traversals — DFS

- Inorder — $L \cdot \text{Root} \cdot R$
- Preorder — $\text{Root} \cdot L \cdot R$
- Postorder — $L \cdot R \cdot \text{Root}$
- Iterative versions use an explicit stack

Traversals — BFS

- Level Order — queue based; one level per outer iteration
- Variants — Zigzag, Level Order II (bottom-up)

Tree Views

- Left / Right view — first node seen per level
- Top / Bottom view — first / last horizontal-distance node
- Vertical Order — group by column index using a TreeMap

Recursive recipes

- $\text{Height} = 1 + \max(\text{height}(L), \text{height}(R))$
- $\text{Diameter} = \max(\text{height}(L) + \text{height}(R) + 1)$
- Path Sum — DFS carrying running sum
- LCA — return the node where matches from both subtrees meet

Topic 24

Binary Tree — Problems

- 258. Inorder Traversal [Easy] — DFS
- 259. Preorder Traversal [Easy] — DFS
- 260. Postorder Traversal [Easy] — DFS
- 261. Maximum Depth of Binary Tree [Easy] — DFS
- 262. Level Order Traversal [Easy] — BFS
- 263. Level Order Traversal II [Easy] — BFS
- 264. Symmetric Tree [Easy] — DFS
- 265. Path Sum [Easy] — DFS
- 266. Populate Next Right Pointers [Medium] — BFS
- 267. Diameter of Binary Tree [Medium] — DFS
- 268. Balanced Binary Tree [Easy] — DFS
- 269. Same Tree [Easy] — DFS
- 270. Zigzag Level Order Traversal [Medium] — BFS
- 271. Left View of Binary Tree [Medium] — BFS / DFS
- 272. Right View of Binary Tree [Medium] — BFS / DFS
- 273. Top View of Binary Tree [Medium] — BFS + Map
- 274. Bottom View of Binary Tree [Medium] — BFS + Map
- 275. Vertical Order Traversal [Hard] — BFS + Map
- 276. Sum of Longest Bloodline [Medium] — DFS
- 277. Lowest Common Ancestor [Medium] — DFS
- 278. Path Sum II [Medium] — DFS + Backtracking
- 279. Path Sum III [Medium] — Prefix Sum + DFS
- 280. Construct Tree (Preorder + Inorder) [Medium] — Recursion
- 281. Burning Tree [Hard] — BFS + Mapping
- 282. Morris Traversal [Hard] — Threaded Tree

Topic 25

Binary Search Tree (BST)

BST property

- left subtree < node < right subtree (for every node)
- Inorder traversal of a BST yields sorted order

Core operations — avg $O(\log N)$, worst $O(N)$ skewed

- Search — compare and recurse on the matching side
- Insert — walk to the right empty spot and attach a leaf
- Delete — three cases: no child / one child / two children (replace with inorder successor or predecessor)

Common Patterns

- Floor / Ceil — binary-search-style descent
- Kth smallest / largest — counted inorder traversal
- LCA — first node where the two values diverge across its key
- Validate BST — pass (min, max) bounds down the recursion

Topic 25

BST — Problems

- 283. Validate Binary Search Tree [Medium] — DFS (Bounds Check)
- 284. Search in BST [Easy] — BST Traversal
- 285. Kth Smallest in BST [Medium] — Inorder Traversal
- 286. Floor and Ceil in BST [Medium] — BST Traversal
- 287. Delete Node in BST [Medium] — BST Manipulation
- 288. Predecessor and Successor [Medium] — BST Traversal
- 289. Lowest Common Ancestor in BST [Easy] — BST Property
- 290. Two Sum IV (BST) [Easy] — Hashing / Inorder
- 291. Convert Sorted Array to BST [Easy] — Divide & Conquer

Topic 26

Balanced Trees (AVL, Red-Black)

Why balance?

- Skewed BST degrades to $O(N)$ per operation
- Self-balancing trees keep height $O(\log N)$

AVL Tree

- Strictly balanced — $|\text{height}(L) - \text{height}(R)| \leq 1$ for every node
- Re-balance with rotations after insert / delete
- Rotations — LL, RR, LR, RL
- Faster lookups; more rotations on writes than Red-Black

Red-Black Tree

- Each node is red or black; root and NIL leaves are black
- No two red nodes adjacent; every root-to-NIL path has the same black count
- Looser balance than AVL → fewer rotations on writes
- Java's TreeMap / TreeSet are Red-Black trees

When to use what

- AVL — read-heavy workloads (lookups dominate)
- Red-Black — mixed / generic library choice

Topic 27

Binary Heap

Heap shape

- Complete binary tree — every level full except possibly the last; last level fills from the left
- Stored as an array — root at index 1 $\rightarrow$ left = $2i$, right = $2i+1$

Heap-order property

- Max-heap — parent $\geq$ children (root holds the max)
- Min-heap — parent $\leq$ children (root holds the min)

Operations

- insert / offer — append + bubble up — $O(\log N)$
- extract / poll — swap root & last + heapify down — $O(\log N)$
- peek — $O(1)$ · build heap from array — $O(N)$
- Heap Sort — $O(N \log N)$, in-place

PriorityQueue (Java)

- Default is a min-heap
- Max-heap — `new PriorityQueue<>(Collections.reverseOrder())`
- Custom order via Comparator

Comparator vs Comparable

- `Comparable<T>` — natural ordering via `compareTo(T o)`
- `Comparator<T>` — external ordering via `compare(T a, T b)` — any number per type

Topic 27

Heap — Problems

- 292. Last Stone Weight [Easy] — Max Heap
- 293. Kth Largest Element in Array [Medium] — Heap
- 294. K Closest Points to Origin [Medium] — Heap
- 295. Top K Frequent Elements [Medium] — Heap + Hashing
- 296. Top K Frequent Words [Medium] — Heap + Sorting
- 297. Sort Characters By Frequency [Medium] — Heap + Hashing

Topic 28

Sliding Window Technique

Idea

- Maintain a contiguous window $[l..r]$ over the array / string
- Move r to expand, move l to shrink
- Aggregate (sum / count / map) updated in $O(1)$ per step — overall $O(N)$

Fixed-size window

- Window size = K is given
- Slide by one — add $a[r]$, remove $a[r - K]$

Variable-size window

- Grow r while valid; once invalid, shrink l until valid again
- Track best answer at every valid state

When to reach for it

- Subarray / substring problems with 'longest', 'shortest', 'count', 'at most K ' constraints
- Frequencies that change as window moves — pair with HashMap

Topic 28

Sliding Window — Problems

- 298. Max Sum Subarray of size K [Easy] — Fixed Window
- 299. Maximum Points You Can Obtain [Medium] — Sliding Window
- 300. Count Occurrence of Anagram [Medium] — Sliding Window + Hashing
- 301. Largest Subarray with Sum K [Medium] — Variable Window
- 302. Pick Toys [Medium] — Variable Window
- 303. Longest Substring Without Repeating Characters [Medium] — Sliding Window + Hashing
- 304. Minimum Window Substring [Hard] — Sliding Window + Hashing

Topic 29

Graph

Representation

- Adjacency Matrix — $V \times V \cdot O(V^2)$ space · $O(1)$ edge check
- Adjacency List — Map / List of lists · $O(V+E)$ space (preferred for sparse)

Traversal

- BFS — queue based · shortest path on unweighted graphs
- DFS — stack / recursion · useful for cycle / connectivity / topo sort

Cycle Detection

- Undirected — visited + parent (DFS) or BFS
- Directed — recursion stack (DFS) or in-degree count (Kahn's BFS)

Topological Sort (DAG only)

- DFS — post-order reverse
- Kahn's BFS — repeatedly pick zero in-degree vertices

Shortest Path Algorithms

- Dijkstra — non-negative weights · $O((V+E) \log V)$ with PQ
- Bellman-Ford — handles negative edges, detects negative cycles · $O(V \cdot E)$
- Floyd-Warshall — all-pairs · $O(V^3)$
- BFS — unweighted single-source

Minimum Spanning Tree (MST)

- Connect all V vertices with $V-1$ edges of minimum total weight
- Prim's — grow MST from any vertex using PQ · $O(E \log V)$
- Kruskal's — sort edges + Union-Find · $O(E \log E)$

Disjoint Set Union (DSU)

- Operations — find(x), union(a , b)
- Optimisations — union by rank / size, path compression → near $O(\alpha(N))$

Topic 29

Graph — Problems

- 305. BFS Traversal [Easy] — BFS
- 306. DFS Traversal [Easy] — DFS
- 307. Cycle Detection (Undirected) [Medium] — BFS / DFS
- 308. Cycle Detection (Directed) [Medium] — DFS
- 309. Topological Sort (DFS) [Medium] — DFS
- 310. Kahn's Algorithm [Medium] — BFS
- 311. Topological Sort (Kahn's) [Medium] — BFS
- 312. Cycle Detection (Kahn's) [Medium] — BFS
- 313. Keys and Rooms [Medium] — DFS
- 314. All Paths from Source to Target [Medium] — DFS
- 315. Number of Islands [Medium] — DFS / BFS
- 316. Dijkstra Algorithm [Medium] — Shortest Path
- 317. Bellman-Ford Algorithm [Medium] — Shortest Path
- 318. Prim's Algorithm [Medium] — MST
- 319. Kruskal's Algorithm [Medium] — MST
- 320. Floyd-Warshall Algorithm [Hard] — All-Pairs Shortest Path
- 321. Find City with Smallest Neighbors [Medium] — Graph + Shortest Path
- 322. Flood Fill [Easy] — BFS / DFS
- 323. Clone Graph [Medium] — DFS / BFS
- 324. Number of Provinces [Medium] — DFS / BFS
- 325. Course Schedule [Medium] — Topological Sort
- 326. Cheapest Flights Within K Stops [Hard] — BFS / Dijkstra

Topic 30

Dynamic Programming (DP)

Why DP?

- Overlapping subproblems + Optimal substructure
- Avoid recomputation by caching subproblem answers

Two Flavours

- Memoization (Top-Down) — recursion + cache
- Tabulation (Bottom-Up) — iterative table fill
- Often $O(\text{states} \times \text{transitions})$ time — states determine space

How to design a DP

- Define state(s) clearly — what does $dp[i][\dots]$ represent?
- Write the recurrence (transition) using smaller states
- Set base cases
- Decide iteration order so dependencies are filled first

Common Patterns

- 1D — Fibonacci, Climbing Stairs, House Robber
- Knapsack — 0/1 (pick / skip), Unbounded (pick any count)
- String DP — LCS, LIS, Edit Distance, Supersequence
- Partition DP — MCM, Palindrome Partitioning, Burst Balloons
- Grid DP — unique paths, min path sum

Space Optimisations

- Most 2D DPs depend only on previous row → reduce to two 1D arrays
- Some 1D DPs depend only on last 2 values → constant space

Topic 30

DP — Problems

- 327. Fibonacci Number [Easy] — DP (1D)
- 328. Climbing Stairs [Easy] — DP (1D)
- 329. Min Cost Climbing Stairs [Easy] — DP (1D)
- 330. Jump Game [Medium] — Greedy / DP
- 331. 0/1 Knapsack [Medium] — DP (Knapsack)
- 332. Subset Sum Problem [Medium] — DP (Subset)
- 333. Partition Equal Subset Sum [Medium] — DP (Subset)
- 334. Perfect Sum Problem [Medium] — DP (Count Subsets)
- 335. Unbounded Knapsack [Medium] — DP (Knapsack)
- 336. Rod Cutting [Medium] — DP (Unbounded Knapsack)
- 337. Coin Change II [Medium] — DP (Count Ways)
- 338. Coin Change [Medium] — DP (Min Coins)
- 339. Longest Common Subsequence [Medium] — DP (Strings)
- 340. Longest Common Substring [Medium] — DP (Strings)
- 341. Print Longest Common Subsequence [Medium] — DP (Reconstruction)
- 342. Shortest Common Supersequence [Medium] — DP (Strings)
- 343. Print Shortest Common Supersequence [Medium] — DP (Reconstruction)
- 344. Longest Palindromic Subsequence [Medium] — DP (Strings)
- 345. Minimum Deletions to Make String Palindrome [Medium] — DP (Strings)
- 346. Matrix Chain Multiplication [Hard] — DP (Partition)
- 347. Palindrome Partitioning [Hard] — DP (Partition)
- 348. Longest Palindromic Substring [Medium] — DP / Expand
- 349. Egg Dropping Puzzle [Hard] — DP (Partition)
- 350. Scramble String [Hard] — DP (Strings)

Topic 31

Trie (Prefix Tree)

Structure

- Tree where each edge represents a character
- Each node holds — children[ALPHABET], isEnd flag
- Root represents the empty prefix

Operations ($L = \text{word length}$)

- insert(word) — walk / create nodes — $O(L)$
- search(word) — exact match — $O(L)$
- startsWith(prefix) — prefix match — $O(L)$
- delete(word) — unset isEnd and prune empty branches

Why Trie?

- Excellent for prefix queries, autocomplete, dictionaries
- Combines with Backtracking (Word Search II) or DP (Word Break)
- Bit-Trie variant solves Maximum XOR style problems

Trade-offs

- Heavy memory — one child slot per alphabet letter per node
- Use HashMap children for large / dynamic alphabets

Topic 31

Trie — Problems

- 351. Implement Trie (Prefix Tree) [Medium] — Trie
- 352. Longest Common Prefix [Easy] — Trie / String
- 353. Word Break [Medium] — Trie + DP
- 354. Word Search II [Hard] — Trie + Backtracking
- 355. Maximum XOR of Two Numbers in Array [Medium] — Trie (Bit Manipulation)

Topic 32

How to use these notes

- Read the concept block of a topic first — internalise vocabulary.
- Pick problems from the list top-down — Easy → Medium → Hard.
- After solving, note the Pattern tag — repetition cements pattern recognition.
- Revisit one topic per sitting — bullets are designed for 5–10 minute revision.
- Use the difficulty colour as a spaced-repetition signal — re-do Hard ones twice a week.

All 355 problems from the syllabus are indexed. Concept blocks are revision-grade — they pair best with hands-on practice.